

Data Structure – Solved Questions

Q1: Search in a linked queue and move element to front

```
void SearchAndMoveToFront(Queue *q, int key) {
    Node *prev = NULL, *cur = q->front;
    while (cur != NULL && cur->data != key) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL || cur == q->front) return;
    prev->next = cur->next;
    if (cur == q->rear) q->rear = prev;
    cur->next = q->front;
    q->front = cur;
}
```

Q2: Balanced parentheses using stack

```
int isBalanced(char exp[]) {
    Stack s; CreateStack(&s);
    for (int i = 0; exp[i] != '\0'; i++) {
        if (exp[i] == '(') Push(&s, '(');
        else if (exp[i] == ')') {
            if (IsEmpty(s)) return 0;
            Pop(&s);
        }
    }
    return IsEmpty(s);
}
```

Q3: Enqueue positive front, negative rear

```
void EnqueueSpecial(Queue *q, int x) {
    Node *n = malloc(sizeof(Node));
    n->data = x; n->next = NULL;
    if (IsEmpty(*q)) {
        q->front = q->rear = n;
    } else if (x >= 0) {
        n->next = q->front;
        q->front = n;
    } else {
        q->rear->next = n;
        q->rear = n;
    }
}
```

Q4: Count full nodes in tree

```
int CountFullNodes(Node *root) {
    if (!root) return 0;
    if (root->left && root->right)
        return 1 + CountFullNodes(root->left) + CountFullNodes(root->right);
    return CountFullNodes(root->left) + CountFullNodes(root->right);
}
```

Q5: Remove odd elements from linked list

```
void RemoveOdd(ListType *L) {
```

```

Node *cur = L->head, *prev = NULL;
while (cur) {
    if (cur->data % 2 != 0) {
        Node *tmp = cur;
        if (!prev) L->head = cur->next;
        else prev->next = cur->next;
        cur = cur->next;
        free(tmp);
    } else {
        prev = cur;
        cur = cur->next;
    }
}
}

```

Q6: Delete smallest node in BST

```

Node* DeleteMin(Node *root) {
    if (!root) return NULL;
    if (!root->left) {
        Node *tmp = root->right;
        free(root);
        return tmp;
    }
    root->left = DeleteMin(root->left);
    return root;
}

```

Q7: Delete leaf nodes in BST

```

Node* DeleteLeaves(Node *root) {
    if (!root) return NULL;
    if (!root->left && !root->right) {
        free(root);
        return NULL;
    }
    root->left = DeleteLeaves(root->left);
    root->right = DeleteLeaves(root->right);
    return root;
}

```

Q8: Delete largest node in BST

```

Node* DeleteMax(Node *root) {
    if (!root) return NULL;
    if (!root->right) {
        Node *tmp = root->left;
        free(root);
        return tmp;
    }
    root->right = DeleteMax(root->right);
    return root;
}

```

Q9: Swap first and last elements in queue

```

void SwapFirstLast(Queue *q) {
    if (!q->front || q->front == q->rear) return;
    int temp = q->front->data;
    q->front->data = q->rear->data;
    q->rear->data = temp;
}

```

Q10: Maximum value in BST

```
int FindMax(Node *root) {
    while (root->right) root = root->right;
    return root->data;
}
```

Q11: Minimum value in BST

```
int FindMin(Node *root) {
    while (root->left) root = root->left;
    return root->data;
}
```

Q12: Count leaf nodes in BST

```
int CountLeaves(Node *root) {
    if (!root) return 0;
    if (!root->left && !root->right) return 1;
    return CountLeaves(root->left) + CountLeaves(root->right);
}
```

Q13: Check if two BSTs are identical

```
int AreIdentical(Node *t1, Node *t2) {
    if (!t1 && !t2) return 1;
    if (!t1 || !t2) return 0;
    return (t1->data == t2->data &&
            AreIdentical(t1->left, t2->left) &&
            AreIdentical(t1->right, t2->right));
}
```